REVISION CHEAT SHEET: DEPENDENCY INJECTION

Dependency Injection and Service Lifetimes

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. Why Dependency Injection Matters
2. Dependency Injection, Dependency Inversion, and IoC
Dependency Injection
Dependency Inversion Principle
Inversion of Control
Mental Model
3. Constructor Injection
4. Constructor Injection vs Property Injection
5. The Composition Root
6. IServiceCollection
7. IServiceProvider
8. ServiceDescriptor
9. Basic Registration Forms
Interface to Implementation
Concrete Type Registration
Factory Registration
Existing Instance Registration
10. The Three Core Service Lifetimes
11. Transient Lifetime
Transient Example
When Transient Fits
When Transient May Be a Poor Fit
12. Scoped Lifetime
Scoped Request Flow
When Scoped Fits
13. Singleton Lifetime
Singleton Examples
14. Singleton Thread Safety
Unsafe Singleton
15. Lifetime Comparison
16. Lifetime Decision Question
17. Lifetime Dependency Rule
18. Captive Dependency
19. Lifetime Dependency Matrix
20. Scope Validation
21. Root Provider vs Scope Provider
22. Explicitly Creating a Scope
23. BackgroundService Lifetime
Correct BackgroundService
24. Why Background Workers Need Scopes
25. DbContext Lifetime
26. DbContext Is Not Thread-Safe
27. Service Disposal
Example
28. Do Not Dispose Injected Dependencies Yourself
29. Disposable Transient Trap
30. Async Service Construction
Better Async Initialization
31. Service Locator Anti-Pattern
Better
32. When IServiceProvider Is Legitimate
33. Avoid BuildServiceProvider During Registration
Better
34. Multiple Implementations
Resolving One
Example
35. TryAdd
36. TryAddEnumerable
37. Keyed Services
38. FromKeyedServices
When Keyed Services Are Useful

When Keyed Services Can Become a Smell
39. Open Generic Registration
40. Be Careful with Generic Repository Abstractions
41. ActivatorUtilities
42. Circular Dependencies
Fixing Circular Dependencies
43. Factory Pattern with DI
44. DI Factory Should Remain Fast
45. IHttpClientFactory and DI
46. Typed HttpClient Lifetime Caveat
Example Singleton HTTP Consumer
47. Scope Is Not Automatically a Thread Boundary
48. DI and Security
49. Do Not Cache HttpContext in Singleton State
50. Memory Impact of Singleton
51. Performance and DI
52. DI and Object Allocation
53. Singleton Is Not a Performance Shortcut
54. DI and Unit Testing
55. Integration Testing Registrations
56. Do Not Mock Everything
57. Options and DI
58. Choosing a Lifetime — Fundraising Examples
GiftReferenceFormatter
GiftRepository
GiftApplicationService
SystemClock
FundRuleSnapshot
CurrentTenantContext
FundraisingDbContext
59. Lifetime Decision Flow
60. Complete Fundraising DI Setup
61. Complete Request Object Graph
62. Background Outbox Flow
63. Common Mistake — Everything Is Singleton
64. Common Mistake — Everything Is Transient
65. Common Mistake — Singleton Depends on Scoped
66. Common Mistake — Injecting IServiceProvider Everywhere
67. Common Mistake — BuildServiceProvider in Program.cs
68. Common Mistake — Disposing DI-Owned Service
69. Common Mistake — Root Resolve of Scoped Service
70. Common Mistake — Singleton Mutable User State
71. Common Mistake — Expensive Work in DI Factory
72. Common Mistake — Typed HttpClient Captured by Singleton
73. Common Mistake — DbContext in Parallel
74. Common Mistake — Circular Dependency Hidden with IServiceProvider
75. Common Mistake — Interface for Every Class
76. Best Practices
77. Practical Exercise 1 — Choose the Lifetime
78. Practical Exercise 2 — Repository Lifetime
79. Practical Exercise 3 — Clock Lifetime
80. Practical Exercise 4 — Find Captive Dependency
81. Practical Exercise 5 — Background Worker
82. Practical Exercise 6 — Multiple Strategies
83. Practical Exercise 7 — Keyed Service
84. Practical Exercise 8 — Open Generic
85. Practical Exercise 9 — Disposal Ownership
86. Practical Exercise 10 — DbContext Parallelism
87. Practical Exercise 11 — Root Provider
88. Practical Exercise 12 — Diagnose Constructor Size
89. Exactly 25 Interview Questions
1. What is Dependency Injection?
2. What is the difference between DI, DIP, and IoC?
3. What are IServiceCollection and IServiceProvider?
4. What are the three main DI lifetimes?
5. What is a transient service?
6. What is a scoped service?
7. What is a singleton service?
8. What is a captive dependency?
9. Can a singleton depend on a scoped service?
10. Can a singleton depend on a transient service?
11. What is scope validation?
12. Why is DbContext usually scoped?
13. Is DbContext thread-safe?
14. Who disposes DI-created services?
15. What happens when a disposable transient is resolved from the root provider?
16. Why should BuildServiceProvider normally not be called during service registration?
17. What is the Service Locator anti-pattern?

18. How does the built-in DI container handle multiple implementations?
19. What are keyed services?
20. What is an open generic DI registration?
21. What is ActivatorUtilities?
22. How should a BackgroundService use scoped services?
23. Why use IHttpClientFactory?
24. Is DI automatically thread-safe?
25. How would you choose lifetimes in the Fundraising Platform?
90. Quick Revision
91. Lifetime Diagram
92. Dependency Lifetir
93. Complete End-to-End Fundraising Example
Program.cs
Controller
Application Service
Repository
Background Worker
94. Final End-to-End Concept Flow
95. Final Mental Model

DEPENDENCY INJECTION

1. Why Dependency Injection Matters

A modern ASP.NET Core application is normally built from many collaborating objects:

EXAMPLE / FLOW

```
Controller
    ↓
Application Service
    ↓
Repository
    ↓
DbContext
    ↓
SQL Server
```

The same application may also use:

EXAMPLE / FLOW

```
Logging
HTTP clients
Caching
Configuration
Messaging
Email
Background workers
Authorization services
```

The important design question is:

> **Who creates these objects, how do they receive their dependencies, and how long should each object live?**

Without a dependency-injection system, classes often create their own dependencies:

C#

```
public sealed class GiftService
{
    private readonly GiftRepository _repository = new();

    private readonly EmailReceiptSender _receiptSender = new();

    public async Task ProcessAsync(
        Gift gift,
```

```
        CancellationToken cancellationToken)
    {
        await _repository.SaveAsync(
            gift,
            cancellationToken);

        await _receiptSender.SendAsync(
            gift,
            cancellationToken);
    }
}
```

DEPENDENCY INJECTION

This creates several problems:

EXAMPLE / FLOW

```
GiftService controls dependency construction

Concrete implementations are hard-coded

Testing becomes harder

Lifetimes become hidden

Resource disposal becomes harder

Replacing infrastructure requires changing business code
```

Dependency injection changes the relationship:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;
    private readonly IReceiptSender _receiptSender;

    public GiftService(
        IGiftRepository repository,
        IReceiptSender receiptSender)
    {
        _repository = repository;

        _receiptSender = receiptSender;
    }
}
```

Now `GiftService` declares:

EXAMPLE / FLOW

```
"I require these capabilities."
```

but does not decide:

EXAMPLE / FLOW

```
which implementation to construct

how to construct it

how long it should live

who disposes it
```

Those decisions move to the application's **composition root**, normally `Program.cs`.

2. Dependency Injection, Dependency Inversion, and IoC

DEPENDENCY INJECTION

These concepts are related but different.

Dependency Injection

Dependency Injection means:

> ⓘ An object receives the dependencies it needs from outside rather than constructing them internally.

Example:

```
C#
public GiftService(
    IGiftRepository repository,
    IReceiptSender receiptSender)
```

Dependencies are injected into the constructor.

Dependency Inversion Principle

From Topic 31:

> ⓘ High-level business policy should depend on abstractions instead of low-level implementation details.

Example:

```
EXAMPLE / FLOW
GiftApplicationService
        ↓
IGiftRepository
        ↑
GiftRepository
```

The application layer knows:

```
EXAMPLE / FLOW
IGiftRepository
```

not:

```
EXAMPLE / FLOW
EF Core details
SQL connection details
```

Inversion of Control

IoC is broader.

Without IoC:

```
EXAMPLE / FLOW
GiftService
    ↓
creates repository
```

```
    ↓
creates email sender
    ↓
controls object graph
```

DEPENDENCY INJECTION

With IoC:

EXAMPLE / FLOW

```
ASP.NET Core container
    ↓
creates repository
creates sender
creates service
creates controller
```

The framework controls object construction.

Mental Model

EXAMPLE / FLOW

```
DIP
    →
Design principle

DI
    →
How dependencies are supplied

IoC Container
    →
Framework mechanism that constructs
and connects objects
```

3. Constructor Injection

Constructor injection is the normal/default approach in ASP.NET Core.

Example:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;
    private readonly ILogger<GiftService> _logger;

    public GiftService(
        IGiftRepository repository,
        ILogger<GiftService> logger)
    {
        _repository = repository;

        _logger = logger;
    }
}
```

Benefits:

EXAMPLE / FLOW

```
Dependencies are explicit

Object cannot be created incorrectly
```

```
Dependencies can be readonly

Unit testing is easy

Service requirements are visible in IntelliSense
```

ASP.NET Core's built-in container primarily uses constructor injection. The runtime selects a public constructor whose parameters can be satisfied by DI; when several constructors exist, constructor selection follows resolvability rules and ambiguous applicable constructors cause problems.

4. Constructor Injection vs Property Injection

Property injection would look like:

C#

```
public IGiftRepository Repository
{
    get;
    set;
}
```

The built-in .NET DI container does not provide general property injection as one of its standard features. Microsoft's guidance recommends the built-in container for most applications and notes that property injection is one of the specialized features that may motivate a third-party container.

Constructor injection is normally safer because:

EXAMPLE / FLOW

```
dependency is mandatory

object is valid after construction

property cannot accidentally remain null
```

5. The Composition Root

The composition root is where object relationships are configured.

In an ASP.NET Core application this is normally:

EXAMPLE / FLOW

```
Program.cs
```

Example:

C#

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        GiftService>();

builder.Services
    .AddSingleton<
```

```
        IClock,
        SystemClock>();
```


Then:

```C#
WebApplication app = builder.Build();
```

After the container is built, the runtime can construct object graphs.

6. IServiceCollection

During application startup:

```C#
builder.Services
```

is an:

```C#
IServiceCollection
```

Think of it as:

```EXAMPLE / FLOW
A collection of service registrations
```

Each registration describes:

```EXAMPLE / FLOW
Service contract

Implementation

Lifetime

Optional factory/instance/key
```

Internally, the collection contains `ServiceDescriptor` entries. Microsoft's registration documentation explicitly describes `IServiceCollection` as a collection of service descriptors.

7. IServiceProvider

After the host/container is built, registrations become resolvable through:

```C#
IServiceProvider
```

Conceptually:

```EXAMPLE / FLOW
IServiceCollection
    ↓ Build
IServiceProvider
    ↓ Resolve
Object graph
```

Example:

```
C#
GiftService service =
    serviceProvider
        .GetRequiredService<GiftService>();
```

In normal application code, however, prefer constructor injection rather than manually resolving services.

8. ServiceDescriptor

Conceptually a registration contains:

```
EXAMPLE / FLOW
Service type
Implementation type / factory / instance
Lifetime
```

Example:

```
C#
ServiceDescriptor descriptor =
    new(
        typeof(IGiftRepository),
        typeof(GiftRepository),
        ServiceLifetime.Scoped);
```

Then:

```
C#
builder.Services.Add(
    descriptor);
```

Usually you use the simpler extension methods:

```
C#
AddTransient
AddScoped
AddSingleton
```

instead of constructing `ServiceDescriptor` manually.

9. Basic Registration Forms

Interface to Implementation

```
C#
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

Meaning:

```
EXAMPLE / FLOW
When IGiftRepository is requested
    ↓
create/use GiftRepository
according to scoped lifetime
```

Concrete Type Registration

```C#
builder.Services
    .AddScoped<
        GiftService>();
```

Equivalent conceptually to:

```C#
builder.Services
    .AddScoped<
        GiftService,
        GiftService>();
```

Microsoft documents this equivalence for implementation-only registrations.

Factory Registration

```C#
builder.Services
    .AddSingleton<IClock>(
        serviceProvider =>
            new SystemClock());
```

Factories are useful when construction requires:

EXAMPLE / FLOW

```
Configuration values

Other registered services

Custom constructor parameters
```

Example:

```C#
builder.Services
    .AddSingleton<ReceiptPolicy>(
        serviceProvider =>
        {
            IConfiguration configuration =
                serviceProvider
                    .GetRequiredService<IConfiguration>();

            decimal threshold =
                configuration
                    .GetValue<decimal>(
                        "Receipts:MajorGiftThreshold");

            return new ReceiptPolicy(
                threshold);
        });
```

Existing Instance Registration


```C#
ReceiptPolicy policy =
    new(
        10_000m);

builder.Services
    .AddSingleton<
        ReceiptPolicy>(
            policy);
```

Important ownership distinction:

> ⓘ If you give the container an already-created instance, the container does not own its creation/ disposal in the same way as instances it constructs itself.

Microsoft's registration table distinguishes container-created singleton registrations from externally supplied instances: externally supplied instances don't receive automatic disposal by the container.

10. The Three Core Service Lifetimes

The built-in container provides three main lifetimes:

EXAMPLE / FLOW

```
Transient
Scoped
Singleton
```

Their meaning is fundamental to ASP.NET Core architecture.

11. Transient Lifetime

Register:

```C#
builder.Services
    .AddTransient<
        IGiftFormatter,
        GiftFormatter>();
```

Transient means:

> ⓘ A new instance is created whenever the service is requested/resolved.

Conceptually:

EXAMPLE / FLOW

```
Request service
    ↓
Instance A

Request service again
    ↓
Instance B

Request service again
```

```
    ↓
Instance C
```

Transient Example

```C#
public sealed class GiftFormatter :
    IGiftFormatter
{
    private readonly Guid _id = Guid.NewGuid();

    public Guid Id =>
        _id;
}
```

Registration:

```C#
builder.Services
    .AddTransient<
        IGiftFormatter,
        GiftFormatter>();
```

If two different consumers request `IGiftFormatter`:

```EXAMPLE / FLOW
Consumer A → GiftFormatter #1
Consumer B → GiftFormatter #2
```

When Transient Fits

Good candidates:

```EXAMPLE / FLOW
Lightweight stateless services

Formatting services

Mapping helpers

Small business calculators

Short-lived operations
```

Example:

```C#
public sealed class GiftReferenceFormatter
{
    public string Format(
        long giftId)
        =>
        $"GFT-{giftId:D10}";
}
```

This has:

EXAMPLE / FLOW

```
no shared mutable state
cheap construction
```

Transient can be appropriate.

When Transient May Be a Poor Fit

Avoid creating transient services that are:

EXAMPLE / FLOW

```
Very expensive to initialize

Holding expensive native resources

Large object graphs

Intended to share state
```

Also remember:

> ⓘ If a transient is injected into a singleton, the singleton keeps that particular transient instance for the singleton's entire lifetime.

The registration says:

EXAMPLE / FLOW

```
Transient
```

but the object graph determines its effective lifetime after injection.

Microsoft specifically warns that a transient captured by a singleton may need singleton-like thread-safety and behaves as a captive dependency.

12. Scoped Lifetime

Register:

C#

```
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

Scoped means:

> ⓘ One service instance is created per DI scope.

In ASP.NET Core:

EXAMPLE / FLOW

```
one HTTP request
```

normally corresponds to:

EXAMPLE / FLOW

```
one DI scope
```

Therefore:

EXAMPLE / FLOW

```
Request A
    ↓
GiftRepository A

Request B
    ↓
GiftRepository B
```

Within Request A:

EXAMPLE / FLOW

```
Controller
Service
Other scoped dependency
```

requesting the same scoped registration receive the same scoped instance.

Microsoft's lifetime guidance states that request-processing applications dispose scoped services at the end of the request and that `AddScoped` represents one instance per scope.

Scoped Request Flow

EXAMPLE / FLOW

```
HTTP Request A
      ↓
Request Scope A
      ├─ GiftController
      ├─ GiftService
      ├─ GiftRepository
      └─ FundraisingDbContext

HTTP Request B
      ↓
Request Scope B
      ├─ GiftController
      ├─ GiftService
      ├─ GiftRepository
      └─ FundraisingDbContext
```

Instances do not cross request scopes unless your design deliberately stores them somewhere longer-lived.

When Scoped Fits

Common scoped services:

EXAMPLE / FLOW

```
DbContext

Repository

Unit of Work

Request-oriented application service
```

```
Request-specific context/accessor state
```

13. Singleton Lifetime

Register:

C#

```
builder.Services
    .AddSingleton<
        IClock,
        SystemClock>();
```

Singleton means:

> One instance is reused for the lifetime of the root service provider/application.

Conceptually:

EXAMPLE / FLOW

```
Application starts

          Singleton #1
        ↙      ↓       ↘
Request A Request B Request C

Application stops
```

Microsoft documents singleton services as being created on first request—or supplied directly—and reused for all subsequent resolutions, with container-created singleton services disposed when the root service provider is disposed.

Singleton Examples

Potential candidates:

EXAMPLE / FLOW

```
Stateless clock service

Thread-safe metadata provider

Immutable configuration snapshot holder

Expensive thread-safe shared service
```

Example:

C#

```
public sealed class SystemClock :
    IClock
{
    public DateTimeOffset UtcNow =>
        DateTimeOffset.UtcNow;
}
```

This contains no mutable request state.

Singleton is reasonable.

14. Singleton Thread Safety

A singleton may be called concurrently by many requests:

EXAMPLE / FLOW

```
Request A ─┐
Request B ─┼─→ Same Singleton
Request C ─┘
```

Therefore:

> **Singleton services must be thread-safe when used concurrently.**

Microsoft explicitly requires singleton services to be thread-safe and notes that DI container resolution being thread-safe does not make your service object's own mutable state thread-safe.

Unsafe Singleton

C#

```
public sealed class TenantState
{
    public long CurrentTenantId
    {
        get;
        set;
    }
}
```

Registered:

C#

```
builder.Services
    .AddSingleton<TenantState>();
```

Request A:

EXAMPLE / FLOW

```
TenantId = 10
```

Request B:

EXAMPLE / FLOW

```
TenantId = 20
```

Both manipulate the same object.

This could cause:

EXAMPLE / FLOW

```
cross-request contamination

cross-tenant data exposure

race conditions
```

Never keep per-request security context in ordinary singleton mutable fields.

15. Lifetime Comparison

Lifetime	Instance creation	Typical ASP.NET Core lifetime	Common uses
Transient	Each resolution	Very short	Stateless lightweight helpers
Scoped	Once per scope	Usually one request	DbContext, repository, use-case services
Singleton	Once per root container	Entire app lifetime	Thread-safe shared services

16. Lifetime Decision Question

Ask:

EXAMPLE / FLOW

```
Does this object contain request-specific state?
```

Yes:

EXAMPLE / FLOW

```
Usually Scoped
```

Ask:

EXAMPLE / FLOW

```
Does this object need shared application-wide state?
```

Yes:

EXAMPLE / FLOW

```
Maybe Singleton
```

but only if:

EXAMPLE / FLOW

```
thread-safe
bounded
lifetime is intentional
```

Ask:

EXAMPLE / FLOW

```
Is it lightweight and stateless?
```

Yes:

EXAMPLE / FLOW

```
Transient may be simplest
```

17. Lifetime Dependency Rule

A crucial rule is:

> **Long-lived objects should not directly capture shorter-lived objects whose lifetime semantics they would violate.**

The classic error:

EXAMPLE / FLOW

```
Singleton
    ↓
Scoped
```

Example:

C#

```
public sealed class GiftMetricsCache
{
    private readonly FundraisingDbContext _dbContext;

    public GiftMetricsCache(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }
}
```

Registration:

C#

```
builder.Services
    .AddSingleton<GiftMetricsCache>();

builder.Services
    .AddDbContext<
        FundraisingDbContext>();
```

Problem:

EXAMPLE / FLOW

```
GiftMetricsCache lives all application lifetime

DbContext is supposed to live one request/scope
```

The singleton captures it and keeps it far beyond its intended scope.

This is the classic **captive dependency**. Microsoft's DI guidance explicitly describes singleton → scoped capture as a lifetime misconfiguration.

18. Captive Dependency

Definition:

> ⓘ A longer-lived service keeps a shorter-lived dependency alive longer than intended.

Example:

EXAMPLE / FLOW

```
Singleton
    ↓
Scoped Repository
    ↓
Scoped DbContext
```

The scoped instances effectively become application-long objects.

Possible consequences:

EXAMPLE / FLOW

```
Wrong request state

Thread-safety failures

DbContext concurrency bugs

Memory retention

Disposed-object surprises

Cross-user data leakage
```

19. Lifetime Dependency Matrix

A practical model:

Consumer	Singleton dependency	Scoped dependency	Transient dependency
Singleton	✅	❌ direct capture	⚠️ captured for singleton lifetime
Scoped	✅	✅	✅
Transient inside valid scope	✅	✅	✅

The transient column deserves care:

EXAMPLE / FLOW

```
Transient does not mean
"it disappears automatically."
```

If a singleton receives a transient in its constructor:

EXAMPLE / FLOW

```
singleton owns reference
    ↓
transient survives as long as singleton
```

That transient may therefore also need thread-safety.

20. Scope Validation

ASP.NET Core can detect common lifetime errors.

Microsoft states that the default provider in development validates that:

EXAMPLE / FLOW

```
Scoped services are not resolved from root provider

Scoped services are not injected into singletons
```

Example bad configuration:

C#

```
builder.Services
    .AddScoped<
        FundraisingDbContext>();

builder.Services
```

```
    .AddSingleton<
        GiftCache>();
```


where:

```
C#
public GiftCache(
    FundraisingDbContext dbContext)
```

may result in an error similar to:

```
EXAMPLE / FLOW
Cannot consume scoped service
from singleton
```

That is a useful failure.

Do not disable validation merely to hide a lifetime design bug.

21. Root Provider vs Scope Provider

After building the application:

```
EXAMPLE / FLOW
Root ServiceProvider
```

lives for the application.

Request processing creates:

```
EXAMPLE / FLOW
Request IServiceScope
```

Conceptually:

```
EXAMPLE / FLOW
Root Provider
    ├─ Singleton A
    ├─ Singleton B
    │
    ├─ Request Scope #1
    │     ├─ Scoped A1
    │     └─ Scoped B1
    │
    └─ Request Scope #2
          ├─ Scoped A2
          └─ Scoped B2
```

Scoped services should normally be resolved from:

```
EXAMPLE / FLOW
scope provider
```

not the root provider.

22. Explicitly Creating a Scope

Sometimes a singleton legitimately needs to perform work using scoped services.

Do not inject the scoped service directly.

Inject:

```C#
IServiceScopeFactory
```

Example:

```C#
public sealed class GiftMaintenanceWorker
{
    private readonly IServiceScopeFactory _scopeFactory;

    public GiftMaintenanceWorker(
        IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    public async Task RunOnceAsync(
        CancellationToken cancellationToken)
    {
        await using AsyncServiceScope scope =
            _scopeFactory
                .CreateAsyncScope();

        IGiftRepository repository =
            scope.ServiceProvider
                .GetRequiredService<
                    IGiftRepository>();

        await repository
            .RunMaintenanceAsync(
                cancellationToken);
    }
}
```

Now lifetime flow is:

```EXAMPLE / FLOW
Singleton
    ↓
Create new scope
    ↓
Resolve scoped repository
    ↓
Use repository
    ↓
Dispose scope
    ↓
Scoped objects disposed
```

23. BackgroundService Lifetime

A service registered with:

```C#
AddHostedService<T>()
```

is registered as a singleton.

Therefore this is wrong:

C#

DEPENDENCY INJECTION

```
public sealed class GiftWorker :
    BackgroundService
{
    private readonly FundraisingDbContext _dbContext;

    public GiftWorker(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }
}
```

because:

EXAMPLE / FLOW

```
BackgroundService = singleton

DbContext = scoped
```

Microsoft's current guidance explicitly says hosted services are singleton and that scoped work should be performed by creating an explicit async scope using `IServiceScopeFactory`.

Correct BackgroundService

C#

```
public sealed class GiftWorker :
    BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public GiftWorker(
        IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (
            !stoppingToken
                .IsCancellationRequested)
        {
            await using AsyncServiceScope scope =
                _scopeFactory
                    .CreateAsyncScope();

            IOutboxProcessor processor =
                scope.ServiceProvider
                    .GetRequiredService<
                        IOutboxProcessor>();

            await processor
                .ProcessAsync(
                    stoppingToken);

            await Task.Delay(
                TimeSpan.FromSeconds(5),
                stoppingToken);
        }
```

```
    }
}
```


Each iteration gets:

EXAMPLE / FLOW

```
fresh scoped services
```

24. Why Background Workers Need Scopes

Suppose:

EXAMPLE / FLOW

```
Worker lives 30 days
```

but each batch should have a fresh:

EXAMPLE / FLOW

```
DbContext
Repository
Unit of Work
```

Creating one scope per operation/batch gives:

EXAMPLE / FLOW

```
Batch 1
    ↓
DbContext #1
    ↓ disposed

Batch 2
    ↓
DbContext #2
    ↓ disposed
```

instead of:

EXAMPLE / FLOW

```
one DbContext used forever
```

25. DbContext Lifetime

EF Core's:

C#

```
AddDbContext<TContext>()
```

registers the context as **scoped by default**.

That aligns well with:

EXAMPLE / FLOW

```
one HTTP request
    ≈
one unit of work
```

Microsoft's EF Core guidance states that `AddDbContext` uses scoped lifetime by default and that `DbContext` is designed to represent a short unit of work.

Example:

```C#
builder.Services
    .AddDbContext<
        FundraisingDbContext>(
            options =>
            {
                options
                    .UseSqlServer(
                        connectionString);
            });
```

26. DbContext Is Not Thread-Safe

Do not share one context concurrently.

Bad:

```C#
Task<Gift?> giftTask =
    dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId
                == giftId);

Task<Campaign?> campaignTask =
    dbContext.Campaigns
        .SingleOrDefaultAsync(
            campaign =>
                campaign.CampaignId
                == campaignId);

await Task.WhenAll(
    giftTask,
    campaignTask);
```

Same context, simultaneous operations.

EF Core explicitly states that the same `DbContext` does not support multiple parallel operations or concurrent use from different threads.

Prefer:

```C#
Gift? gift =
    await dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId
                ==
                giftId,
            cancellationToken);

Campaign? campaign =
    await dbContext.Campaigns
        .SingleOrDefaultAsync(
            campaign =>
                campaign.CampaignId
                ==
```

```
            campaignId,
        cancellationToken);
```

DEPENDENCY INJECTION

when using one context.

27. Service Disposal

When the DI container creates a service that implements:

C#

```
IDisposable
```

or:

C#

```
IAsyncDisposable
```

the container normally owns its cleanup.

Microsoft's guidance says:

EXAMPLE / FLOW

```
Transient/scoped disposable services
    →
disposed when owning scope ends

Singleton disposable services
    →
disposed when root container shuts down
```

Example

C#

```
public sealed class GiftFileWriter :
    IDisposable
{
    public void Dispose()
    {
        // Clean resources.
    }
}
```

Registered:

C#

```
builder.Services
    .AddScoped<
        GiftFileWriter>();
```

ASP.NET Core will dispose the instance when the request scope ends.

28. Do Not Dispose Injected Dependencies Yourself

Bad:

C#

```
public sealed class GiftService :
    IDisposable
{
```

```
    private readonly GiftRepository _repository;

    public GiftService(
        GiftRepository repository)
    {
        _repository = repository;
    }

    public void Dispose()
    {
        _repository.Dispose();
    }
}
```

DEPENDENCY INJECTION

If the repository was supplied by DI:

EXAMPLE / FLOW

```
the container owns it
```

The consumer should normally not dispose it.

Microsoft specifically says that receiving an `IDisposable` through DI does not require the receiver to dispose that dependency itself.

29. Disposable Transient Trap

Suppose:

C#

```
builder.Services
    .AddTransient<
        ExpensiveDisposable>();
```

Then root-level code repeatedly does:

C#

```
ExpensiveDisposable service =
    app.Services
        .GetRequiredService<
            ExpensiveDisposable>();
```

The root container tracks container-created disposable transient instances so it can dispose them later.

If thousands are resolved from root:

EXAMPLE / FLOW

```
root provider
    ↓
retains disposable transients
    ↓
application lifetime
```

This can look like a memory leak.

Microsoft explicitly warns that disposable transients resolved from the top-level/root container can be captured until root disposal.

Use a scope or explicit factory/lifetime strategy.

30. Async Service Construction

The built-in container does not support:

EXAMPLE / FLOW

```
async constructors
```

and DI factories should remain synchronous.

Bad:

C#

```
builder.Services
    .AddSingleton<GiftCache>(
        serviceProvider =>
        {
            return LoadCacheAsync()
                .Result;
        });
```

This can block or deadlock.

Microsoft's DI guidance explicitly states that async/Task-based service resolution is not supported and warns against sync-over-async factories.

Better Async Initialization

Construct synchronously:

C#

```
public sealed class GiftCache
{
    public async Task InitializeAsync(
        CancellationToken cancellationToken)
    {
        // Load data.
    }
}
```

Then initialize during:

EXAMPLE / FLOW

```
application startup

hosted startup workflow

first explicit async operation
```

depending on design.

31. Service Locator Anti-Pattern

Bad:

C#

```
public sealed class GiftService
{
    private readonly IServiceProvider _provider;

    public GiftService(
```

```
        IServiceProvider provider)
    {
        _provider = provider;
    }

    public Task ProcessAsync()
    {
        IGiftRepository repository =
            _provider
                .GetRequiredService<
                    IGiftRepository>();

        // ...
        return Task.CompletedTask;
    }
}
```

DEPENDENCY INJECTION

Now constructor says:

```
EXAMPLE / FLOW
GiftService requires IServiceProvider
```

instead of revealing:

```
EXAMPLE / FLOW
GiftService requires IGiftRepository
```

Dependency is hidden.

Microsoft's DI guidance explicitly recommends avoiding the service-locator pattern when constructor injection can be used instead.

Better

```
C#
public GiftService(
    IGiftRepository repository)
```

Now dependencies are:

```
EXAMPLE / FLOW
explicit
```

32. When IServiceProvider Is Legitimate

`IServiceProvider` is not forbidden.

Legitimate infrastructure scenarios include:

```
EXAMPLE / FLOW
Creating an explicit scope

Framework integration

Runtime plugin infrastructure

Generic factories where dependency types
are truly dynamic
```

The issue is not:

EXAMPLE / FLOW

```
IServiceProvider exists
```

The issue is:

EXAMPLE / FLOW

```
business classes using it as a hidden
dependency locator
```

33. Avoid BuildServiceProvider During Registration

Bad:

C#

```
IServiceProvider temporary =
    builder.Services
        .BuildServiceProvider();

IConfiguration configuration =
    temporary
        .GetRequiredService<
            IConfiguration>();
```

This builds a separate provider.

Potential problems:

EXAMPLE / FLOW

```
Duplicate singleton instances

Different lifetimes

Extra disposal complexity

Inconsistent dependency graph
```

Microsoft explicitly advises against calling `BuildServiceProvider` while registering services. Use registration overloads that receive `IServiceProvider` instead.

Better

C#

```
builder.Services
    .AddSingleton<ReceiptPolicy>(
        serviceProvider =>
        {
            IConfiguration configuration =
                serviceProvider
                    .GetRequiredService<
                        IConfiguration>();

            return new ReceiptPolicy(
                configuration
                    .GetValue<decimal>(
                        "Receipt:Threshold"));
        });
```

34. Multiple Implementations

Suppose:

```C#
public interface IReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Implementations:

EXAMPLE / FLOW

```
EmailReceiptSender

PrintReceiptSender

SmsReceiptSender
```

Register:

```C#
builder.Services
    .AddScoped<
        IReceiptSender,
        EmailReceiptSender>();

builder.Services
    .AddScoped<
        IReceiptSender,
        PrintReceiptSender>();

builder.Services
    .AddScoped<
        IReceiptSender,
        SmsReceiptSender>();
```

Resolving One

```C#
IReceiptSender sender
```

resolves the **last registered implementation**.

Resolving:

```C#
IEnumerable<IReceiptSender>
```

returns all registrations in registration order.

Microsoft documents exactly this behavior.

Example

C#

```
public sealed class ReceiptProcessor
{
    private readonly
        IReadOnlyList<IReceiptSender>
        _senders;

    public ReceiptProcessor(
        IEnumerable<IReceiptSender> senders)
    {
        _senders = senders.ToList();
    }
}
```

This enables:

EXAMPLE / FLOW

```
strategy-style selection
```

without manually creating implementations.

35. TryAdd

Libraries often should not overwrite an application registration.

Example:

C#

```
services
    .TryAddSingleton<
        ISystemClock,
        SystemClock>();
```

Meaning:

EXAMPLE / FLOW

```
Register SystemClock only if
ISystemClock is not already registered
```

Microsoft provides:

EXAMPLE / FLOW

```
TryAddTransient
TryAddScoped
TryAddSingleton
```

for this purpose.

36. TryAddEnumerable

For library extensions supporting multiple implementations:

C#

```
services.TryAddEnumerable(
    ServiceDescriptor
        .Singleton<
```

```
        IReceiptRule,
        DefaultReceiptRule>());
```

DEPENDENCY INJECTION

This helps avoid accidentally adding the same implementation repeatedly.

Microsoft documents `TryAddEnumerable` specifically for multi-registration scenarios and library authors.

37. Keyed Services

Current .NET DI supports keyed services.

Suppose two cache implementations share one interface:

C#

```
public interface IGiftCache
{
    GiftSummary? Find(
        long giftId);
}
```

Register:

C#

```
builder.Services
    .AddKeyedSingleton<
        IGiftCache,
        MemoryGiftCache>(
            "memory");

builder.Services
    .AddKeyedSingleton<
        IGiftCache,
        DistributedGiftCache>(
            "distributed");
```

The built-in container supports:

EXAMPLE / FLOW

```
AddKeyedTransient
AddKeyedScoped
AddKeyedSingleton
```

and keys can be objects, not only strings.

38. FromKeyedServices

Consumer:

C#

```
public sealed class GiftLookupService
{
    private readonly IGiftCache _cache;

    public GiftLookupService(
        [FromKeyedServices("memory")]
        IGiftCache cache)
    {
        _cache = cache;
    }
}
```

ASP.NET Core also supports keyed resolution in controller/action scenarios using `FromKeyedServices`.

When Keyed Services Are Useful

Examples:

EXAMPLE / FLOW

```
Multiple payment gateways

Multiple storage providers

Multiple cache implementations

Different tenant-independent algorithms

Multiple notification transports
```

When Keyed Services Can Become a Smell

Bad:

EXAMPLE / FLOW

```
"email"
"sms"
"pdf"
"postgres"
"mode17"
```

scattered throughout application code.

If every business class contains key strings:

EXAMPLE / FLOW

```
DI container configuration has leaked
into business logic
```

A domain-oriented strategy/resolver abstraction may be clearer.

39. Open Generic Registration

Suppose:

C#

```
public interface IRepository<TEntity>
    where TEntity : class
{
}
```

Implementation:

C#

```
public sealed class Repository<TEntity> :
    IRepository<TEntity>
    where TEntity : class
{
}
```

Rather than registering:

EXAMPLE / FLOW

```
IRepository<Gift>
IRepository<Campaign>
IRepository<Fund>
...
```

you can register an open generic:

C#

```
builder.Services.AddScoped(
    typeof(IRepository<>),
    typeof(Repository<>));
```

Then the container can create:

EXAMPLE / FLOW

```
IRepository<Gift>
IRepository<Campaign>
IRepository<Fund>
```

when requested.

The built-in registration APIs support `Type`-based service/implementation registrations, including the form used for generic type definitions.

40. Be Careful with Generic Repository Abstractions

Just because DI supports:

C#

```
IRepository<TEntity>
```

does not mean every project should use a generic repository.

In your Fundraising Platform, ask:

EXAMPLE / FLOW

```
Does generic repository add a useful abstraction?

Or does it merely wrap DbSet<TEntity>
without meaningful domain behavior?
```

Architecture first.

DI registration convenience second.

41. ActivatorUtilities

Sometimes you need to construct an object that:

EXAMPLE / FLOW

```
is not registered as a service
```

but should still receive registered dependencies.

Use:

C#

```
ActivatorUtilities.CreateInstance<T>()
```

Example:

C#
```
GiftImporter importer =
    ActivatorUtilities
        .CreateInstance<
            GiftImporter>(
                serviceProvider,
                importFileName);
```

The direct parameter:

EXAMPLE / FLOW
```
importFileName
```

comes from your code.

Other constructor dependencies can come from DI.

Microsoft documents `ActivatorUtilities.CreateInstance` as creating a type with constructor values supplied directly and/or by an `IServiceProvider`.

42. Circular Dependencies

Suppose:

EXAMPLE / FLOW
```
GiftService
    ↓
ReceiptService
    ↓
GiftService
```

Constructors:

C#
```
public GiftService(
    ReceiptService receiptService)
```

C#
```
public ReceiptService(
    GiftService giftService)
```

The container cannot create either first.

This is a circular dependency.

Usually it indicates:

EXAMPLE / FLOW
```
Responsibilities are tangled

Wrong abstraction boundary

Missing lower-level shared service

Orchestration is in wrong class
```

Fixing Circular Dependencies

Instead of:

```
EXAMPLE / FLOW
GiftService ↔ ReceiptService
```

consider:

```
EXAMPLE / FLOW
GiftWorkflow
   ↓       ↓
GiftService ReceiptService
```

or extract shared behavior:

```
EXAMPLE / FLOW
GiftService
    ↓
IGiftReader

ReceiptService
    ↓
IGiftReader
```

depending on responsibility.

Do not automatically solve every cycle with:

```
EXAMPLE / FLOW
IServiceProvider
Lazy<T>
factory indirection
```

because that may hide the design problem rather than fix it.

43. Factory Pattern with DI

Sometimes runtime selection is legitimate.

Example:

```
C#
public interface IReceiptSenderFactory
{
    IReceiptSender Get(
        ReceiptType type);
}
```

Implementation:

```
C#
public sealed class ReceiptSenderFactory :
    IReceiptSenderFactory
{
    private readonly
        IReadOnlyList<IReceiptSender>
        _senders;

    public ReceiptSenderFactory(
```

```
        IEnumerable<IReceiptSender> senders)
    {
        _senders = senders.ToList();
    }

    public IReceiptSender Get(
        ReceiptType type)
    {
        return _senders
            .Single(
                sender =>
                    sender.CanHandle(
                        type));
    }
}
```

DEPENDENCY INJECTION

This preserves:

```
EXAMPLE / FLOW
explicit typed dependency
```

instead of giving business logic raw `IServiceProvider`.

44. DI Factory Should Remain Fast

Registration factory:

```
C#
services.AddSingleton<T>(
    serviceProvider =>
    {
        // ...
    });
```

should generally perform synchronous object construction.

Do not:

```
EXAMPLE / FLOW
Perform network call

Execute SQL query

Block on async

Run expensive startup workflow
```

inside arbitrary DI factories.

Microsoft recommends keeping DI factories fast and synchronous.

45. IHttpClientFactory and DI

For outgoing HTTP integrations:

```
EXAMPLE / FLOW
Payment API
Receipt API
Constituent API
CRM API
```

prefer managed HTTP client patterns rather than creating/discarding arbitrary `HttpClient` instances.

Registration:


```C#
builder.Services
    .AddHttpClient<
        PaymentGatewayClient>(
            client =>
            {
                client.BaseAddress =
                    new Uri(
                        "https://payments.example/");
            });
```

Typed client:

```C#
public sealed class PaymentGatewayClient
{
    private readonly HttpClient _httpClient;

    public PaymentGatewayClient(
        HttpClient httpClient)
    {
        _httpClient = httpClient;
    }
}
```

`IHttpClientFactory` integrates client configuration, DI, logging, handler lifetime management, and named/typed clients.

46. Typed HttpClient Lifetime Caveat

Typed clients created with:

```C#
AddHttpClient<TClient>()
```

are transient.

Do not casually capture a typed client in a singleton.

Microsoft warns that capturing a typed client in a singleton can prevent expected handler/DNS refresh behavior and recommends using the named-client factory approach or carefully configured handler lifetime when a singleton needs HTTP clients.

Example Singleton HTTP Consumer

Prefer:

```C#
public sealed class GlobalStatusMonitor
{
    private readonly IHttpClientFactory _httpClientFactory;

    public GlobalStatusMonitor(
        IHttpClientFactory httpClientFactory)
    {
        _httpClientFactory = httpClientFactory;
    }
```

```
    public async Task CheckAsync(
        CancellationToken cancellationToken)
    {
        HttpClient client =
            _httpClientFactory
                .CreateClient(
                    "StatusApi");

        await client.GetAsync(
            "/health",
            cancellationToken);
    }
}
```

47. Scope Is Not Automatically a Thread Boundary

A scoped service means:

EXAMPLE / FLOW

```
one instance per scope
```

It does not mean:

EXAMPLE / FLOW

```
thread-safe
```

A request can launch concurrent work:

C#

```
Task first = DoFirstAsync();

Task second = DoSecondAsync();

await Task.WhenAll(
    first,
    second);
```

If both use the same scoped mutable service:

EXAMPLE / FLOW

```
same object may be accessed concurrently
```

Lifetime and thread safety are separate concerns.

48. DI and Security

Dependency lifetimes can create serious security bugs.

Bad singleton:

C#

```
public sealed class CurrentTenant
{
    public long TenantId
    {
        get;
        set;
    }
}
```

Registered:

DEPENDENCY INJECTION

C#
```
AddSingleton<CurrentTenant>();
```

Request A:

EXAMPLE / FLOW
```
Tenant 10
```

Request B:

EXAMPLE / FLOW
```
Tenant 20
```

They share the same object.

Result can be:

EXAMPLE / FLOW
```
cross-tenant state contamination
```

Per-request context should normally be scoped/request-bound.

49. Do Not Cache HttpContext in Singleton State

Bad:

C#
```
public sealed class GlobalUserState
{
    public HttpContext? Current
    {
        get;
        set;
    }
}
```

This is dangerous because:

EXAMPLE / FLOW
```
HttpContext belongs to one request

Singleton belongs to all requests
```

Instead, inject:

EXAMPLE / FLOW
```
IHttpContextAccessor
```

where truly appropriate, or better, extract only needed request information into an explicit scoped/application abstraction.

50. Memory Impact of Singleton

A singleton is effectively rooted for the application lifetime.

If it references:

EXAMPLE / FLOW

```
10 MB cache
large graph
event subscribers
request object
```

those objects may remain alive as long as the singleton references them.

Microsoft's DI guidance explicitly warns that singleton memory is retained until application/container shutdown and that singleton object graphs can have significant memory impact.

51. Performance and DI

The built-in DI container is optimized for common application workloads.

Do not avoid constructor injection because of fear of:

EXAMPLE / FLOW

```
one interface call
```

Typical ASP.NET Core performance costs are more likely dominated by:

EXAMPLE / FLOW

```
SQL
HTTP calls
Serialization
File I/O
Large allocations
Poor queries
```

rather than ordinary DI resolution.

However, excessive object graphs can matter.

Example:

EXAMPLE / FLOW

```
Controller
 → Service
 → Coordinator
 → Handler
 → Provider
 → Resolver
 → Factory
 → Adapter
 → Wrapper
```

If each exists only because:

EXAMPLE / FLOW

```
"more DI is better"
```

the design is overcomplicated.

52. DI and Object Allocation

Transient lifetime creates instances frequently.

Suppose:

EXAMPLE / FLOW

```
1000 requests/second
```

and each request resolves:

EXAMPLE / FLOW

```
40 transient services
```

that can mean substantial object allocation.

This is not automatically bad.

Modern .NET handles short-lived allocations efficiently.

But avoid large/expensive transient graphs unless necessary.

53. Singleton Is Not a Performance Shortcut

Do not change:

EXAMPLE / FLOW

```
Scoped
```

to:

EXAMPLE / FLOW

```
Singleton
```

just to avoid allocations.

A singleton changes semantics:

EXAMPLE / FLOW

```
shared state
thread safety
resource lifetime
memory retention
```

Correct lifetime matters more than saving a few object allocations.

54. DI and Unit Testing

Constructor injection makes testing simple.

Production:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;

    public GiftService(
        IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Test:

C#

```
IGiftRepository repository = new FakeGiftRepository();

GiftService service =
    new(
        repository);
```

DEPENDENCY INJECTION

No container is required in most unit tests.

That is important:

> **Unit tests normally test the class directly, not the DI container.**

55. Integration Testing Registrations

Integration tests may replace registrations with:

EXAMPLE / FLOW

```
Fake service

Test database

Stub external integration
```

Example conceptually:

C#

```
services.RemoveAll<
    IPaymentGateway>();

services.AddSingleton<
    IPaymentGateway,
    FakePaymentGateway>();
```

The point is that dependency boundaries allow the application's composition root to vary without changing business logic.

56. Do Not Mock Everything

Bad test:

EXAMPLE / FLOW

```
Mock ILogger
Mock Mapper
Mock Clock
Mock Builder
Mock Factory
Mock Resolver
Mock Provider
Mock Adapter
```

for a tiny business rule.

If testing requires 15 mocks:

EXAMPLE / FLOW

```
the class may have too many responsibilities
```

or the test may be too implementation-focused.

57. Options and DI

Configuration should normally not be registered as arbitrary mutable objects only to make DI work.

Microsoft's DI guidelines recommend using the Options pattern for configuration rather than treating configuration values as random service-container data holders.

Conceptually:

```C#
builder.Services
    .Configure<ReceiptOptions>(
        builder.Configuration
            .GetSection(
                "Receipt"));
```

Then services can depend on:

```EXAMPLE / FLOW
IOptions<T>
IOptionsSnapshot<T>
IOptionsMonitor<T>
```

depending on required configuration behavior.

Detailed Options patterns are architectural support, not a separate lifetime replacement.

58. Choosing a Lifetime — Fundraising Examples

GiftReferenceFormatter

```C#
public sealed class GiftReferenceFormatter
{
    public string Format(
        long giftId)
        =>
        $"GFT-{giftId:D10}";
}
```

Stateless and cheap:

```EXAMPLE / FLOW
Transient
```

is reasonable.

GiftRepository

Depends on request `DbContext`:

```EXAMPLE / FLOW
Scoped
```

GiftApplicationService

Coordinates one request/use case and depends on repository:

EXAMPLE / FLOW

```
Scoped
```

DEPENDENCY INJECTION

is usually natural.

SystemClock

Stateless:

EXAMPLE / FLOW

```
Singleton
```

is reasonable.

FundRuleSnapshot

If immutable/thread-safe and shared application-wide:

EXAMPLE / FLOW

```
Singleton
```

may be appropriate.

CurrentTenantContext

Contains request-specific tenant/user information:

EXAMPLE / FLOW

```
Scoped
```

not singleton.

FundraisingDbContext

EXAMPLE / FLOW

```
Scoped by AddDbContext
```

by default.

59. Lifetime Decision Flow

EXAMPLE / FLOW

```
Does service contain request-specific state?
    ↓
Yes
    →
Scoped
```

EXAMPLE / FLOW

```
Does service depend on DbContext?
    ↓
Usually Scoped
```

EXAMPLE / FLOW

```
Does service need one thread-safe shared instance
for the whole application?
    ↓
Yes
    →
Singleton
```

EXAMPLE / FLOW

```
Is service lightweight/stateless
and independent?
    ↓
Transient may fit
```

Then ask:

EXAMPLE / FLOW

```
What lifetime do its dependencies have?
```

This question is just as important as:

EXAMPLE / FLOW

```
What lifetime should this service have?
```

60. Complete Fundraising DI Setup

C#

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddControllers();

builder.Services
    .AddDbContext<
        FundraisingDbContext>(
            options =>
            {
                options.UseSqlServer(
                    builder.Configuration
                        .GetConnectionString(
                            "FundraisingDb"));
            });

builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        ICampaignRepository,
        CampaignRepository>();

builder.Services
    .AddScoped<
        CreateGiftService>();

builder.Services
    .AddScoped<
        CompleteGiftService>();
```

```
builder.Services
    .AddTransient<
        IGiftReferenceFormatter,
        GiftReferenceFormatter>();

builder.Services
    .AddSingleton<
        IClock,
        SystemClock>();

builder.Services
    .AddKeyedScoped<
        IReceiptSender,
        EmailReceiptSender>(
            "email");

builder.Services
    .AddKeyedScoped<
        IReceiptSender,
        PrintReceiptSender>(
            "print");

builder.Services
    .AddHttpClient<
        PaymentGatewayClient>(
            client =>
            {
                client.BaseAddress =
                    new Uri(
                        builder.Configuration[
                            "Payments:BaseUrl"]!);
            });

builder.Services
    .AddHostedService<
        OutboxBackgroundService>();
```

61. Complete Request Object Graph

Request:

EXAMPLE / FLOW

```
POST /api/gifts
```

ASP.NET Core creates:

EXAMPLE / FLOW

```
Request Scope
    ↓
GiftController
    ↓
CreateGiftService
    ↓
IGiftRepository
    ↓
GiftRepository
    ↓
FundraisingDbContext
```

Also:

EXAMPLE / FLOW

```
CreateGiftService
    ↓
IClock
    ↓
SystemClock singleton
```

At request completion:

EXAMPLE / FLOW

```
Request scope disposed
    ↓
GiftController released
CreateGiftService released
GiftRepository disposed/released
DbContext disposed
```

But:

EXAMPLE / FLOW

```
SystemClock singleton
```

continues living.

62. Background Outbox Flow

EXAMPLE / FLOW

```
Application Lifetime
    ↓
OutboxBackgroundService singleton
    ↓
Create scope
    ↓
OutboxProcessor scoped
    ↓
FundraisingDbContext scoped
    ↓
Read OutboxMessage
    ↓
Publish
    ↓
Save
    ↓
Dispose scope
```

Next cycle:

EXAMPLE / FLOW

```
new scope
new processor
new DbContext
```

This preserves correct unit-of-work lifetime.

63. Common Mistake — Everything Is Singleton

Bad:

```C#
builder.Services
    .AddSingleton<
        GiftRepository>();

builder.Services
    .AddSingleton<
        FundraisingDbContext>();

builder.Services
    .AddSingleton<
        GiftService>();
```

Problems:

```EXAMPLE / FLOW
DbContext is not thread-safe

Shared tracking state

Concurrent requests share state

Objects live too long

Memory retention
```

64. Common Mistake — Everything Is Transient

Also not ideal.

A request may receive:

```EXAMPLE / FLOW
Repository #1
DbContext #1

Repository #2
DbContext #2

Repository #3
DbContext #3
```

when one consistent request/unit-of-work context was intended.

Service lifetime should express semantics.

65. Common Mistake — Singleton Depends on Scoped

Bad:

```C#
public GlobalGiftCache(
    GiftRepository repository)
```

when:

```EXAMPLE / FLOW
GlobalGiftCache = Singleton
GiftRepository = Scoped
```

This is a captive dependency.

66. Common Mistake — Injecting IServiceProvider Everywhere

Bad:

```C#
public GiftService(
    IServiceProvider provider)
```

then resolving dependencies on demand.

This hides the real contract and becomes service locator.

67. Common Mistake — BuildServiceProvider in Program.cs

Bad:

```C#
var provider =
    builder.Services
        .BuildServiceProvider();
```

during registration.

It can create multiple independent object graphs and singleton instances.

Microsoft explicitly recommends avoiding it.

68. Common Mistake — Disposing DI-Owned Service

Bad:

```C#
using (
    IGiftRepository repository =
        _repository)
{
}
```

if `_repository` was injected.

The DI scope owns its lifetime.

69. Common Mistake — Root Resolve of Scoped Service

Bad:

```C#
IGiftRepository repository =
    app.Services
        .GetRequiredService<
            IGiftRepository>();
```

outside an explicit scope.

This can effectively promote scoped lifetime toward root lifetime and is exactly the type of issue scope validation is intended to catch.

70. Common Mistake — Singleton Mutable User State

Bad:

```C#
public sealed class UserContext
{
    public long UserId
    {
        get;
        set;
    }
}
```

registered singleton.

Every request shares it.

This is a correctness and potentially a security vulnerability.

71. Common Mistake — Expensive Work in DI Factory

Bad:

```C#
services.AddSingleton<GiftCache>(
    provider =>
    {
        // database call
        // HTTP call
        // .Result
        // filesystem scan
    });
```

Factory construction should remain synchronous and reasonably fast.

72. Common Mistake — Typed HttpClient Captured by Singleton

Typed `HttpClient` clients are transient and intended to be short-lived.

Capturing one in a singleton can defeat intended lifetime/DNS refresh behavior.

73. Common Mistake — DbContext in Parallel

Scoped lifetime does not make `DbContext` thread-safe.

Never assume:

```EXAMPLE / FLOW
one request
=
one thread forever
```

EF Core explicitly prohibits concurrent operations on one context.

74. Common Mistake — Circular Dependency Hidden with IServiceProvider

If:

EXAMPLE / FLOW

```
A → B → A
```

DEPENDENCY INJECTION

do not automatically solve it with:

EXAMPLE / FLOW

```
A → IServiceProvider → B
```

You have hidden the cycle.

Revisit responsibilities.

75. Common Mistake — Interface for Every Class

DI does not require:

EXAMPLE / FLOW

```
IGiftFormatter
IGiftMapper
IGiftHelper
IGiftThing
```

for every concrete class.

Interfaces are most useful at:

EXAMPLE / FLOW

```
Architectural boundaries

Replaceable implementations

Meaningful roles
```

76. Best Practices

Prefer constructor injection.

Keep dependencies explicit.

Choose service lifetime based on semantic ownership, not convenience.

Default to scoped for request-oriented services that depend on `DbContext`.

Use singleton only when application-wide lifetime is intentional.

Make singleton state thread-safe.

Never directly inject scoped services into singleton services.

Use `IServiceScopeFactory` when singleton/background infrastructure must perform scoped work.

Let the container dispose services it creates.

Do not manually dispose injected dependencies.

Avoid resolving disposable transient/scoped services from root.

Avoid `BuildServiceProvider` inside registration code.

Avoid service locator.

Keep DI factories synchronous and fast.

Use multiple registrations or keyed services deliberately.

Use `IEnumerable<T>` for strategy collections.

Use keyed services when a key is part of the composition choice, but do not scatter DI keys throughout domain logic.

Use `IHttpClientFactory` for managed outgoing HTTP integration patterns.

Treat `DbContext` as a short-lived unit-of-work object.

Do not share `DbContext` between concurrent operations.

Validate service scopes during development.

77. Practical Exercise 1 — Choose the Lifetime

Service:

```C#
public sealed class GiftReferenceFormatter
{
    public string Format(
        long id)
        =>
        $"GFT-{id}";
}
```

Best reasonable default:

EXAMPLE / FLOW

```
Transient
```

because it is stateless and lightweight.

78. Practical Exercise 2 — Repository Lifetime

```C#
public sealed class GiftRepository
{
    private readonly FundraisingDbContext _dbContext;
}
```

Best normal ASP.NET Core lifetime:

EXAMPLE / FLOW

```
Scoped
```

because `DbContext` is scoped.

79. Practical Exercise 3 — Clock Lifetime

```C#
public sealed class SystemClock
{
    public DateTimeOffset UtcNow =>
        DateTimeOffset.UtcNow;
}
```

Possible lifetime:

EXAMPLE / FLOW

```
Singleton
```

because it is stateless/thread-safe.

80. Practical Exercise 4 — Find Captive Dependency

```C#
services.AddSingleton<
    GiftCache>();

services.AddScoped<
    GiftRepository>();
```

```C#
public GiftCache(
    GiftRepository repository)
```

Problem:

```EXAMPLE / FLOW
Singleton captures scoped repository
```

Fix either:

```EXAMPLE / FLOW
change lifetime
```

or:

```EXAMPLE / FLOW
create explicit scope when required
```

depending on responsibility.

81. Practical Exercise 5 — Background Worker

Wrong:

```C#
public OutboxWorker(
    OutboxProcessor processor)
```

if:

```EXAMPLE / FLOW
OutboxWorker singleton
OutboxProcessor scoped
```

Correct:

```C#
public OutboxWorker(
    IServiceScopeFactory scopeFactory)
```

then create a scope inside each processing cycle.

82. Practical Exercise 6 — Multiple Strategies

```C#
services.AddScoped<
    IReceiptSender,
    EmailReceiptSender>();

services.AddScoped<
    IReceiptSender,
    PrintReceiptSender>();
```

Consumer:

```C#
public ReceiptCoordinator(
    IEnumerable<IReceiptSender> senders)
```

gets both implementations.

83. Practical Exercise 7 — Keyed Service

```C#
services.AddKeyedScoped<
    IReceiptSender,
    EmailReceiptSender>(
        "email");
```

Consumer:

```C#
public GiftProcessor(
    [FromKeyedServices("email")]
    IReceiptSender sender)
```

gets the keyed implementation.

84. Practical Exercise 8 — Open Generic

```C#
services.AddScoped(
    typeof(IRepository<>),
    typeof(Repository<>));
```

Then:

```C#
IRepository<Gift>
```

and:

```C#
IRepository<Campaign>
```

can be closed/resolved from that open registration.

85. Practical Exercise 9 — Disposal Ownership

If the container injects:

```C#
GiftRepository repository
```

should `GiftService` call:

```C#
repository.Dispose();
```

Answer:

EXAMPLE / FLOW

```
No.
```

The owning DI scope/container disposes it.

86. Practical Exercise 10 — DbContext Parallelism

Bad:

```C#
await Task.WhenAll(
    dbContext.Gifts.ToListAsync(),
    dbContext.Campaigns.ToListAsync());
```

Reason:

EXAMPLE / FLOW

```
same DbContext
+
concurrent operations
```

not supported.

87. Practical Exercise 11 — Root Provider

Bad:

```C#
IGiftRepository repo =
    app.Services
        .GetRequiredService<
            IGiftRepository>();
```

Better:

```C#
using IServiceScope scope =
    app.Services
        .CreateScope();

IGiftRepository repo =
    scope.ServiceProvider
        .GetRequiredService<
            IGiftRepository>();
```

when root/startup infrastructure truly needs a scoped service.

88. Practical Exercise 12 — Diagnose Constructor Size

```C#
public GiftService(
    IGiftRepository repo,
    IEmailSender email,
    IPaymentGateway payment,
    IPdfGenerator pdf,
    IAuditWriter audit,
    IMessagePublisher messages,
    ICache cache,
    IFileStore files,
    IReportGenerator reports)
```

Do not immediately:

```EXAMPLE / FLOW
create parameter object
```

Ask:

```EXAMPLE / FLOW
Does GiftService have too many responsibilities?
```

The DI container is exposing a design smell rather than causing it.

89. Exactly 25 Interview Questions

1. What is Dependency Injection?

Dependency Injection is a design technique where an object's dependencies are supplied from outside rather than constructed internally.

Example:

```C#
public GiftService(
    IGiftRepository repository)
```

instead of:

```C#
private readonly GiftRepository _repository = new();
```

The first approach reduces direct coupling and makes dependencies explicit.

2. What is the difference between DI, DIP, and IoC?

DIP is a design principle: depend on abstractions rather than concrete low-level details.

DI is a technique for supplying those dependencies.

IoC is the broader concept where object construction/control is delegated to an external framework/container.

ASP.NET Core's built-in DI container is an IoC mechanism implementing constructor-based dependency injection.

3. What are IServiceCollection and IServiceProvider?

`IServiceCollection` stores service registrations during startup.

`IServiceProvider` is the built container used to resolve object graphs.

Conceptually:

EXAMPLE / FLOW

```
IServiceCollection
    ↓ Build
IServiceProvider
    ↓ Resolve
Service instances
```

`IServiceCollection` is composed of `ServiceDescriptor` registrations.

4. What are the three main DI lifetimes?

They are:

EXAMPLE / FLOW

```
Transient
Scoped
Singleton
```

Transient creates a new instance per resolution.

Scoped creates one instance per scope.

Singleton creates one instance for the root container/application lifetime.

5. What is a transient service?

A transient service is created each time it is requested.

It is typically appropriate for lightweight, stateless services such as formatters or simple calculators.

However, a transient injected into a singleton is retained by that singleton and may therefore effectively live for the entire application lifetime.

6. What is a scoped service?

A scoped service is created once within each DI scope.

In ASP.NET Core, an HTTP request normally establishes a scope, so scoped services are generally reused throughout one request and disposed when the request ends.

7. What is a singleton service?

A singleton service is created once and reused for all subsequent resolutions from the root container.

Singletons live until application/container shutdown and must be thread-safe if accessed concurrently.

8. What is a captive dependency?

A captive dependency occurs when a longer-lived object holds a shorter-lived object beyond its intended lifetime.

Classic example:

EXAMPLE / FLOW

```
Singleton
    ↓
Scoped DbContext
```

The scoped object effectively becomes singleton-like, creating state, concurrency, memory, and disposal problems.

9. Can a singleton depend on a scoped service?

Not directly through normal constructor injection.

The singleton would capture that scoped instance.

If a singleton legitimately needs scoped work, inject `IServiceScopeFactory`, create a scope, resolve the scoped service inside that scope, use it, then dispose the scope.

10. Can a singleton depend on a transient service?

Technically yes, but the injected transient instance is captured for the singleton's lifetime.

That means its effective lifetime becomes long-lived and it may also need thread-safety.

Use this intentionally rather than assuming “transient” guarantees short lifetime.

11. What is scope validation?

Scope validation detects invalid lifetime usage such as:

EXAMPLE / FLOW

```
Scoped service resolved from root

Scoped service injected into singleton
```

The default provider performs these checks in development when using the normal host-building flow.

12. Why is DbContext usually scoped?

A `DbContext` represents a short unit of work and `AddDbContext` registers it as scoped by default.

That maps naturally to one Web API request/unit of work and ensures a separate context for separate request scopes.

13. Is DbContext thread-safe?

No.

EF Core explicitly prohibits concurrent operations on the same `DbContext`.

Always await operations before reusing a context or use separate context instances for genuinely parallel operations.

14. Who disposes DI-created services?

The container disposes services it creates according to their owning lifetime/scope.

Scoped/transient disposables are disposed when their owning scope ends; singleton disposables are disposed with the root container.

Consumers should normally not manually dispose injected dependencies.

15. What happens when a disposable transient is resolved from the root provider?

DEPENDENCY INJECTION

The container may retain it for later disposal until the root provider is disposed.

Repeated root resolution of disposable transients can therefore cause significant retention and appear as a memory leak.

16. Why should BuildServiceProvider normally not be called during service registration?

It creates another service provider/object graph.

This can produce duplicate singleton instances, inconsistent lifetimes, and disposal problems.

Microsoft recommends using registration factory overloads that receive the existing `IServiceProvider` instead.

17. What is the Service Locator anti-pattern?

Service Locator occurs when business classes receive `IServiceProvider` and resolve their dependencies manually:

C#

```
_provider
    .GetRequiredService<
        IGiftRepository>();
```

This hides real dependencies and mixes application logic with container mechanics.

Constructor injection is normally clearer.

18. How does the built-in DI container handle multiple implementations?

If several implementations of one service type are registered:

EXAMPLE / FLOW

```
single T resolution
    →
last registration

IEnumerable<T>
    →
all implementations in registration order
```

19. What are keyed services?

Keyed services allow multiple registrations of the same service type to be distinguished by a key.

Example:

C#

```
services.AddKeyedScoped<
    IReceiptSender,
    EmailReceiptSender>(
        "email");
```

Consumers can select the keyed implementation through keyed DI APIs such as `FromKeyedServices`.

20. What is an open generic DI registration?

An open generic registration maps a generic service definition to a generic implementation:

C#

```
services.AddScoped(
    typeof(IRepository<>),
    typeof(Repository<>));
```

DEPENDENCY INJECTION

The container can then close the generic for:

EXAMPLE / FLOW

```
IRepository<Gift>
IRepository<Campaign>
```

when those types are requested.

21. What is ActivatorUtilities?

`ActivatorUtilities` constructs an object using a combination of:

EXAMPLE / FLOW

```
arguments supplied directly by your code

dependencies resolved from IServiceProvider
```

It is useful when the object itself is not registered as a normal container service.

22. How should a BackgroundService use scoped services?

`BackgroundService` / `IHostedService` registrations are singleton.

Inject `IServiceScopeFactory`, create a new scope for a unit of background work, resolve scoped dependencies from that scope, use them, and dispose the scope.

Do not inject a scoped `DbContext` directly into the singleton worker.

23. Why use IHttpClientFactory?

`IHttpClientFactory` centralizes configuration of outgoing HTTP clients and manages underlying handler lifetimes, logging, and named/typed client patterns.

It helps avoid common connection/DNS lifetime problems associated with poorly managed `HttpClient` creation.

24. Is DI automatically thread-safe?

The built-in provider can safely resolve services concurrently, but this does not make the created service objects thread-safe.

A singleton containing mutable shared state must implement its own synchronization/concurrency design.

25. How would you choose lifetimes in the Fundraising Platform?

A practical starting point is:

EXAMPLE / FLOW

```
FundraisingDbContext
    →
Scoped

GiftRepository
    →
Scoped
```

```
CreateGiftService
    →
Scoped

GiftReferenceFormatter
    →
Transient

SystemClock
    →
Singleton

Immutable Fund rule snapshot
    →
Singleton if thread-safe

CurrentTenantContext
    →
Scoped

OutboxBackgroundService
    →
Singleton hosted service

OutboxProcessor
    →
Scoped and resolved from an explicit scope
```


The governing rule is:

> **Choose the shortest lifetime that correctly represents ownership and sharing requirements, and never let a longer-lived object accidentally capture request-specific or scoped state.**

90. Quick Revision

Concept	Meaning
DI	Dependencies supplied externally
DIP	Depend on abstractions
IoC	Framework controls construction
`IServiceCollection`	Registration collection
`ServiceDescriptor`	Service registration metadata
`IServiceProvider`	Resolves service graphs
Transient	New per resolution
Scoped	One per scope
Singleton	One per application/root provider
Captive dependency	Longer lifetime captures shorter
Scope validation	Detects invalid lifetime usage
Request scope	ASP.NET Core scope per request

Concept	Meaning
`IServiceScopeFactory`	Creates explicit scopes
`CreateAsyncScope`	Async-disposable scope
`BackgroundService`	Singleton hosted service
`AddDbContext`	Scoped context by default
`IEnumerable<T>`	All service implementations
Keyed service	Resolve implementation by key
`TryAdd`	Register only if missing
Open generic	Generic service mapping
`ActivatorUtilities`	DI-assisted activation
Service locator	Hidden runtime resolution anti-pattern
`IHttpClientFactory`	Managed outgoing HTTP clients
Container disposal	DI disposes owned services
Composition root	Central startup registration

DEPENDENCY INJECTION

91. Lifetime Diagram

EXAMPLE / FLOW

```
APPLICATION
│
├── Singleton Service A
│
├── Singleton Service B
│
├── BackgroundService
│
│
├── Request Scope 1
│    │
│    ├── Scoped DbContext #1
│    ├── Scoped GiftRepository #1
│    ├── Scoped GiftService #1
│    │
│    ├── Transient Formatter #1
│    └── Transient Formatter #2
│
├── Request Scope 2
│    │
│    ├── Scoped DbContext #2
│    ├── Scoped GiftRepository #2
│    ├── Scoped GiftService #2
│    │
│    └── Transient Formatter #3
│
└── Application Shutdown
        ↓
    Dispose Singleton Services
```

92. Dependency Lifetime Flow

Safe:

EXAMPLE / FLOW

```
Scoped GiftService
    ↓
Scoped GiftRepository
    ↓
Scoped DbContext
```

Also safe:

EXAMPLE / FLOW

```
Scoped GiftService
    ↓
Singleton SystemClock
```

Potentially dangerous:

EXAMPLE / FLOW

```
Singleton GiftCache
    ↓
Scoped GiftRepository
```

Captive:

EXAMPLE / FLOW

```
Singleton
    ↓
Transient mutable dependency
```

because the transient is now retained by singleton.

93. Complete End-to-End Fundraising Example

Program.cs

C#

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);

builder.Services
    .AddControllers();

builder.Services
    .AddDbContext<
        FundraisingDbContext>(
            options =>
            {
                options.UseSqlServer(
                    builder.Configuration
                        .GetConnectionString(
                            "FundraisingDb"));
            });

builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

```
builder.Services
    .AddScoped<
        CreateGiftService>();

builder.Services
    .AddTransient<
        IGiftReferenceFormatter,
        GiftReferenceFormatter>();

builder.Services
    .AddSingleton<
        IClock,
        SystemClock>();

builder.Services
    .AddHttpClient<
        PaymentGatewayClient>(
            client =>
            {
                client.BaseAddress =
                    new Uri(
                        builder.Configuration[
                            "Payment:BaseUrl"]!);
            });

builder.Services
    .AddHostedService<
        OutboxBackgroundService>();

WebApplication app = builder.Build();

app.MapControllers();

app.Run();
```

Controller

C#

```
[ApiController]
[Route("api/gifts")]
public sealed class GiftController :
    ControllerBase
{
    private readonly CreateGiftService _service;

    public GiftController(
        CreateGiftService service)
    {
        _service = service;
    }

    [HttpPost]
    public async Task<
        ActionResult<CreateGiftResponse>>
        Create(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        CreateGiftResponse response =
            await _service
                .CreateAsync(
                    request,
                    cancellationToken);
```

```
        return Ok(
            response);
    }
}
```

Application Service

C#

```
public sealed class CreateGiftService
{
    private readonly IGiftRepository _repository;
    private readonly IGiftReferenceFormatter _formatter;
    private readonly IClock _clock;

    public CreateGiftService(
        IGiftRepository repository,
        IGiftReferenceFormatter formatter,
        IClock clock)
    {
        _repository = repository;

        _formatter = formatter;

        _clock = clock;
    }

    public async Task<CreateGiftResponse>
        CreateAsync(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        Gift gift =
            Gift.Create(
                request.TenantId,
                request.ConstituentId,
                request.Amount);

        await _repository
            .AddAsync(
                gift,
                cancellationToken);

        return new CreateGiftResponse(
            gift.GiftId,
            _formatter.Format(
                gift.GiftId),
            gift.Amount,
            _clock.UtcNow);
    }
}
```

Repository

C#

```
public sealed class GiftRepository :
    IGiftRepository
{
    private readonly FundraisingDbContext _dbContext;

    public GiftRepository(
```

```
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        _dbContext.Gifts.Add(
            gift);

        await _dbContext
            .SaveChangesAsync(
                cancellationToken);
    }
}
```

Background Worker

C#

```
public sealed class OutboxBackgroundService :
    BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public OutboxBackgroundService(
        IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (
            !stoppingToken
                .IsCancellationRequested)
        {
            await using AsyncServiceScope scope =
                _scopeFactory
                    .CreateAsyncScope();

            OutboxProcessor processor =
                scope.ServiceProvider
                    .GetRequiredService<
                        OutboxProcessor>();

            await processor
                .ProcessAsync(
                    stoppingToken);

            await Task.Delay(
                TimeSpan.FromSeconds(5),
                stoppingToken);
        }
    }
}
```

Register processor:

```C#
builder.Services
    .AddScoped<
        OutboxProcessor>();
```

Now:

```EXAMPLE / FLOW
OutboxBackgroundService
    →
Singleton

Every processing cycle
    →
new scope

OutboxProcessor
    →
Scoped

FundraisingDbContext
    →
Scoped
```

That is the correct lifetime boundary.

94. Final End-to-End Concept Flow

```EXAMPLE / FLOW
Application startup
        ↓
IServiceCollection receives registrations
        ↓
Service descriptors define:
    contract
    implementation
    lifetime
        ↓
Host builds root IServiceProvider
        ↓
Application begins handling requests
```

Request:

```EXAMPLE / FLOW
HTTP Request
    ↓
ASP.NET Core creates request scope
    ↓
Controller requested
    ↓
Container inspects constructor
    ↓
Resolve application service
    ↓
Resolve repository
    ↓
Resolve scoped DbContext
    ↓
Resolve singleton clock
```

```
    ↓
Construct complete object graph
```

During request:

EXAMPLE / FLOW

```
same scoped registration
    →
same scoped instance
```

Different request:

EXAMPLE / FLOW

```
new scope
    →
new scoped instances
```

Singleton:

EXAMPLE / FLOW

```
same application-wide instance
```

Transient:

EXAMPLE / FLOW

```
new resolution
    →
new instance
```

At request completion:

EXAMPLE / FLOW

```
scope disposed
    ↓
scoped/disposable services cleaned up
```

At application shutdown:

EXAMPLE / FLOW

```
root provider disposed
    ↓
container-owned singleton resources cleaned up
```

95. Final Mental Model

Before registering a service, ask five questions:

EXAMPLE / FLOW

```
1. What responsibility does this service have?

2. Who should own its lifetime?

3. Does it contain mutable state?

4. What are the lifetimes of its dependencies?

5. Can multiple requests/threads use it simultaneously?
```

Then choose:

EXAMPLE / FLOW

```
Transient
    →
new per resolution

Scoped
    →
one per logical unit of work/request

Singleton
    →
one shared application-wide instance
```

And always remember:

EXAMPLE / FLOW

```
DI lifetime
    ≠
thread safety
```

EXAMPLE / FLOW

```
Transient
    ≠
guaranteed short lifetime
```

EXAMPLE / FLOW

```
Scoped
    ≠
thread-local
```

EXAMPLE / FLOW

```
Singleton
    ≠
automatically fast
```

EXAMPLE / FLOW

```
IServiceProvider
    ≠
replacement for explicit dependencies
```

EXAMPLE / FLOW

```
DI
    ≠
reason to create interfaces everywhere
```

EXAMPLE / FLOW

```
Dependency Injection
    works best when
    architecture is already well separated.
```

The core lesson is:

Dependency Injection is not primarily about avoiding `new`. It is about making dependencies explicit, moving object construction to the composition root, controlling lifetimes correctly, and preventing business/application code from being coupled to infrastructure creation details. In ASP.NET Core, lifetime selection is as important as the interface design itself: scoped services model request/unit-of-work state, singletons model truly shared thread-safe application state, and transients model short-lived stateless behavior.

Next roadmap topic: Topic 33 — Design Patterns.